CS:344 PYTHON

Augst 2nd 2026

Project milestone 3

My Smart Paycheck Budget Allocator is designed to help users organize their monthly finances by calculating how much money they earn each month and suggesting how to divide it between recurring bills, savings, a rainy-day fund, and a splurge fund. The user enters their take-home paycheck amount, the number of paychecks they receive each month, and each recurring bill. The program then calculates the monthly income, total monthly bills, suggested savings, a $200 rainy-day fund, a splurge fund with a maximum limit of $1,200, and any remaining balance.

The goal of this project is to make budgeting simple and organized while giving users a better understanding of where their money is going each month. Throughout the project, I focused on using functions, loops, lists, conditional statements, and user input to keep the code organized and easier to read. Breaking the program into separate functions also made it easier to test each section individually during debugging.

After testing my program, I found several areas that could be improved. While the calculations work correctly with valid input, the program currently crashes if users enter letters, symbols, commas, or blank values where numbers are expected. I also noticed that the program accepts numbers as bill names, negative paycheck amounts, and zero paychecks, which can produce unrealistic results. Before the final version, I plan to add input validation, improve error messages, allow users to edit values before the final summary, and add additional budgeting features such as bill categories, tax estimation, and other budgeting tools.

Test Case	Purpose	Input	Expected Result
1. Text Instead of Paycheck Amount	Verify the paycheck amount only accepts numbers.	one thousand	The program should reject the input and ask for a valid number.
2. Paycheck Amount with Comma	Verify the program accepts formatted currency.	1,200.50	The program should accept the amount or remove the comma automatically.
3. Blank Paycheck Amount	Verify the paycheck amount cannot be left blank.	Press Enter	The program should display an error and request another input.
4. Negative Paycheck Amount	Verify negative income is rejected.	-1500	The program should reject negative values.
5. Text for Paycheck Count	Verify the paycheck count only accepts whole numbers.	two	The program should reject the text and ask for a number.
6. Zero Paychecks	Verify at least one paycheck is entered.	0	The program should reject zero as an invalid paycheck count.

7. Numeric Bill Name	Verify bill names contain letters instead of numbers.	345	The program should display an invalid bill name message.
8. Incorrect Finish Command	Verify only "done" ends bill entry.	FINISHA	The program should continue asking for bills.
9. Symbols for Paycheck Amount	Verify symbols cannot be entered as income.	$$$$$	The program should reject the symbols and request a valid number.
10. Valid Budget Test	Verify the program works correctly with valid input.	Valid paycheck, paycheck count, bills, and done	The program should calculate and display the complete budget summary correctly.

Testing Environment: I built and tested my Smart Paycheck Budget Allocator on my Mac using Visual Studio Code. I ran the program using Python 3 through the integrated terminal in Visual Studio Code. During testing, I entered both valid and invalid inputs to verify that the budget calculations worked correctly and to identify bugs or unexpected behavior. This testing process helped me find areas that need improvement, such as input validation and error handling, before completing the final version of the project.

Test 1: Text Instead of Paycheck Amount

Why I did it: I wanted to see what would happen if a user accidentally typed words instead of numbers.

Result of debugging: The program crashed with a Value Error because it could not convert text into a number. I need to add input validation to prevent this.

Test 2: Paycheck Amount with Comma

Why I did it: I wanted to see if the program accepted numbers with commas since many people type money that way.

Result of debugging: The program crashed because float() cannot convert numbers with commas. I need to remove commas before converting the input.

Test 3: Blank Paycheck Amount

Why I did it: I wanted to see what happened if the user accidentally pressed Enter without typing anything.

Result of debugging: The program crashed because it could not convert a blank value into a number. I need to check for blank input before processing it.

Test 4: Negative Paycheck Amount

Why I did it: I wanted to make sure users could not enter a negative paycheck amount.

Result of debugging: The program accepted the negative number and created an incorrect budget. I need to prevent negative values.

Test 5: Text for Paycheck Count

Why I did it: I wanted to make sure the paycheck count only accepted whole numbers.

Result of debugging: The program crashed when I entered the word “two.” I need to validate this input just like the paycheck amount.

Test 6: Zero Paychecks

Why I did it: I wanted to see if the program allowed zero paychecks.

Result of debugging: The program accepted zero and created an unrealistic budget. I need to require at least one paycheck.

Test 7: Numeric Bill Name

Why I did it: I wanted to make sure bill names only accepted letters instead of numbers.

Result of debugging: The program accepted numbers as a bill name. I need to validate bill names so they only contain text.

Test 8: Incorrect Finish Command

Why I did it: I wanted to make sure the bill entry loop only stopped when the user typed "done".

Result of debugging: The program worked correctly. It treated the incorrect word as another bill name instead of ending the loop.

Test 9: Symbols for Paycheck Amount

Why I did it: I wanted to see what happened if a user entered only symbols instead of numbers.

Result of debugging: The program crashed because symbols cannot be converted into numbers. I need to validate the input before converting it.

Test 10: Valid Budget Test

Why I did it: I wanted to make sure the program worked correctly when all valid information was entered.

Result of debugging: The program successfully calculated the monthly income, bills, savings, rainy-day fund, splurge fund, and remaining balance. This confirmed that the budget calculations work correctly and that most of the improvements needed are related to user input validation.